



Introdução ao JSON e ao YAML

© 2020, Amazon Web Services, Inc., ou suas afiliadas. Todos os direitos reservados.

Esta lição apresenta a você JavaScript Object Notation (JSON) e YAML Ain't Markup Language (YAML).

JSON e YAML são formatos de dados comumente usados em implantações automatizadas e repetíveis.



O que você aprenderá

Principais pontos da lição

Você aprenderá a:

- Descrever a sintaxe básica de JavaScript Object Notation (JSON) e YAML Ain't Markup Language (YAML)
- Explicar as vantagens e desvantagens dos formatos de dados JSON e YAML

Objetivos:

- Ser capaz de identificar os formatos JSON e YAML
- Entender a sintaxe básica dos dois formatos



■ YAML e JSON: sintaxes para definir a infraestrutura de nuvem

A linguagem da infraestrutura

JSON & YAML: linguagens declarativas para criar infraestruturas de nuvem

- Infraestrutura como código (IaC) é um método para declarar os recursos necessários na nuvem ao usar arquivos de texto
- As sintaxes JavaScript Object Notation (JSON) e YAML Ain't Markup (YAML) são usadas pela AWS para declarar recursos na nuvem
- Permite a definição de infraestruturas simples a complexas no texto
- Compreender as sintaxes de JSON e YAML é necessário para construir a infraestrutura como código

Antes de estudar como definir a infraestrutura no código, é necessário estudar a sintaxe dos dois formatos usados pelo AWS CloudFormation.

Introdução ao JSON

O que é o JSON?

JSON (JavaScript Object Notation)

- É uma sintaxe para armazenar e transportar dados que contém:
 - Pares de chave-valor
 - » {Key1: "Value1", Key2: "Value2"}
 - Matrizes (listas) e outros objetos
 - » {Array: [1, 2, 3]}
- Permite a troca de estruturas de dados complexas em um único documento

Alguns dos serviços da AWS discutidos posteriormente neste módulo, como o AWS CloudFormation, oferecem suporte ao JavaScript Object Notation (JSON) como um formato de dados. O AWS CloudFormation é um serviço para escrever ou alterar modelos que criam e excluem recursos relacionados da AWS juntos como uma unidade (pilha). Para provisionar e configurar seus recursos de pilha, você deve entender os modelos do AWS CloudFormation, que são arquivos de texto formatados em JSON ou YAML. Portanto, nesta seção, você aprenderá os fundamentos do JSON.

JSON é uma sintaxe para armazenar e transportar dados. Ele é um formato baseado em texto e, portanto, é legível por humanos. Os

Alguns dos serviços da AWS discutidos posteriormente neste módulo, como o AWS CloudFormation, oferecem suporte ao JavaScript Object Notation (JSON) como um formato de dados. O AWS CloudFormation é um serviço para escrever ou alterar modelos que criam e excluem recursos relacionados da AWS juntos como uma unidade (pilha). Para provisionar e configurar seus recursos de pilha, você deve entender os modelos do AWS CloudFormation, que são arquivos de texto formatados em JSON ou YAML. Portanto, nesta seção, você aprenderá os fundamentos do JSON.

JSON é uma sintaxe para armazenar e transportar dados. Ele é um formato baseado em texto e, portanto, é legível por humanos. Os documentos JSON são facilmente gravados. Eles armazenam pares de chave-valor e arrays de dados. Um par de chave-valor é um conjunto de dois itens de dados vinculados:

- **Chave** — Identificador exclusivo para um item de dados.
- **Valor** — Dados identificados ou um ponteiro para o local desses dados.

Os dados podem ser estruturados para que você possa armazenar estruturas de dados muito complexas em um único documento JSON. O principal caso de uso para JSON é que ele oferece suporte à simples troca de dados complexos entre sistemas de computador.

Representação de dados

StudentName	Localização	FavoriteSports
María	EUA	Tênis
John	Reino Unido	Cricket
Diego	EUA	Basquete
Kwesi	Índia	Hockey

Dados em linhas e colunas



```
{
  "sport_choices": [
    {
      "StudentName": "María",
      "Location": "USA",
      "FavoriteSport": "Tennis"
    },
    {
      "StudentName": "John",
      "Location": "UK",
      "FavoriteSport": "Cricket"
    },
    {
      "StudentName": "Diego",
      "Location": "USA",
      "FavoriteSport": "Basketball"
    },
    {
      "StudentName": "Kwesi",
      "Location": "India",
      "FavoriteSport": "Hockey"
    }
  ]
}
```

Dados em JSON



Normalmente, você vê dados em um formato colunar, como em uma tabela. No entanto, para armazenar dados de tabela de forma fácil para as máquinas ingerirem, analisarem e distribuírem, eles devem ser armazenados de forma não gráfica.

JSON fornece uma maneira de armazenar objetos de dados em um documento de texto legível por humanos. Conforme ilustrado no exemplo, os dados colunares de uma tabela — que consiste em três colunas e cinco linhas — podem ser armazenados no formato JSON.

Vantagens e desvantagens do JSON

Vantagens:

- Leve (sintaxe e demonstração mínimas) — bom para interfaces de programação de aplicativos (APIs).
- Fácil para humanos lerem e escreverem.
- Fácil para máquinas analisarem e gerarem.

Desvantagens:

- Sem suporte nativo para dados binários (como arquivos de imagem).

Como qualquer padrão, o JSON tem vantagens e desvantagens.

Uma das principais vantagens do JSON é que ele é *leve*. O JSON tem apenas algumas regras de sintaxe que devem ser seguidas para que o documento JSON seja válido. Ele também é uma boa opção para uso com APIs. Na verdade, o JSON é um formato comum de troca de dados para APIs RESTful modernas.

O JSON também é fácil para humanos lerem e é fácil para as máquinas analisarem e gerarem.

No entanto, o JSON também tem algumas desvantagens. Ele é mais detalhado. Como resultado, você pode ter arquivos longos, em que muitas linhas no arquivo

Como qualquer padrão, o JSON tem vantagens e desvantagens.

Uma das principais vantagens do JSON é que ele é *leve*. O JSON tem apenas algumas regras de sintaxe que devem ser seguidas para que o documento JSON seja válido. Ele também é uma boa opção para uso com APIs. Na verdade, o JSON é um formato comum de troca de dados para APIs RESTful modernas.

O JSON também é fácil para humanos lerem e é fácil para as máquinas analisarem e gerarem.

No entanto, o JSON também tem algumas desvantagens. Ele é mais detalhado. Como resultado, você pode ter arquivos longos, em que muitas linhas no arquivo consistem apenas em uma chave de abertura ({} ou chave de fechamento {}). Outra desvantagem do JSON é que ele não tem suporte integrado para armazenar formatos multimídia (como imagens) em JSON.

Blocos de construção JSON: objetos

Como você descreveria esse problema?

Bolo

Chocolate



**Alimenta 8
pessoas**

20 USD

Considere como você descreveria um objeto, como o exemplo mostrado aqui. Quais características ele tem? Quatro características que descrevem esse problema são destacadas.

Blocos de construção JSON: objetos

Você pode estruturar essas informações em um objeto JSON.



Chave **Valor**

```
{  
  "type": "cake",  
  "flavor": "chocolate",  
  "price": "20",  
  "feeds": "8"  
}
```

Aqui, você vê que as características do bolo que foram identificadas agora são capturadas em um objeto JSON.

Blocos de construção JSON: objetos

- As chaves {} atuam como um contêiner.
- O JSON usa estruturas de dados universais:
 - Um objeto é um conjunto não ordenado de pares de chave/valor.
 - Um objeto começa e termina com chaves { }.
 - Cada chave é seguida por dois pontos (:)
 - Cada par de valores-chave é separado por uma vírgula (,).

```
{  
  "type": "cake",  
  "flavor": "chocolate",  
  "price": "20",  
  "feeds": "8"  
}
```

Os objetos JSON estão entre chaves. As estruturas de dados JSON devem seguir regras específicas.

Primeiro, um objeto JSON consiste em um conjunto não ordenado de pares de chave/valor. Observe que um caractere de dois pontos é usado para separar as chaves dos valores, e todas as chaves e valores estão entre aspas. Os pares de chave/valor dentro de um objeto são separados por vírgulas.

Observe que a última par de chave-valor não tem uma vírgula depois dela.

Os objetos JSON estão entre chaves. As estruturas de dados JSON devem seguir regras específicas.

Primeiro, um objeto JSON consiste em um conjunto não ordenado de pares de chave/valor. Observe que um caractere de dois pontos é usado para separar as chaves dos valores, e todas as chaves e valores estão entre aspas. Os pares de chave/valor dentro de um objeto são separados por vírgulas.

Observe que o último par de chave-valor *não* tem uma vírgula depois dele.

No exemplo, suponha que o par de chave-valor *flavor* tenha vindo antes do par de chave-valor *type*. Um programa ou pessoa que recebe esse objeto JSON ainda consideraria o objeto válido.

Se você tivesse outra sobremesa e quisesse descrevê-la usando as mesmas chaves, pode ser algo parecido com este exemplo:

```
{  
  "tipo": "torta",  
  "sabor": "maçã",  
  "preço": "16",  
  "alimenta": "6"
```

```
}
```


Blocos de construção JSON: matrizes (1 de 3)

Quais atributos adicionais você pode usar para descrever esse problema?

Menta



Amoras

Se você quiser ser mais específico sobre esse problema, poderá documentar características adicionais. Por exemplo, tem mirtilos e menta como ingredientes adicionais.

Blocos de construção JSON: matrizes (2 de 3)

Você pode colocar as informações adicionais em um **matriz**.



```
{  
  "type": "cake",  
  "flavor": "chocolate",  
  "price": "20",  
  "feeds": "8",  
  "additional_ingredients": [  
    "blueberries",  
    "mint"  
  ]  
}
```

Aqui, uma matriz **additional_ingredients** foi adicionada ao objeto JSON. Os arrays são formatados de modo diferente dos pares de chave-valor.

As matrizes são usadas quando uma chave pode ter vários valores. Do exemplo, "additional_ingredients" tem os valores "blueberries" e "mint". Alguns outros exemplos de matrizes úteis poderiam uma lista de nomes em um grupo, produtos químicos na fórmula, modelos disponíveis de um fabricante de automóveis e assim por diante.

Blocos de construção JSON: matrizes (3 de 3)

- Colchetes ([]) mantêm matrizes.
- Uma matriz é uma lista ordenada de valores.
- Uma matriz começa com um colchete esquerdo ([) e termina com um colchete direito (]).
- Os valores são separados por uma vírgula (,).
- Os valores podem consistir em vários tipos.
 - String, número, objeto, matriz ou booleano

```
{  
  "type": "cake",  
  "flavor": "chocolate",  
  "price": "20",  
  "feeds": "8",  
  "additional_ingredients": [  
    "blueberries",  
    "mint"  
  ]  
}
```

Para definir uma matriz no JSON, coloque-a entre colchetes.

Um array em JSON é uma coleção ordenada de valores. Ao contrário dos pares de chave-valor na raiz de um objeto JSON, os arrays JSON são ordenados, para que você possa analisá-los por valor de índice. Por exemplo, se você deseja extrair "mint" do objeto de exemplo, consulte **objectName.additional_ingredients[1]**.

Os valores de array podem ser um dos seguintes tipos: string, número, objeto, array ou booleano. O array do exemplo tem dois valores, sendo que ambos são strings (**blueberries** e **mint**). No entanto, em documentos JSON reais, qualquer

Para definir uma matriz no JSON, coloque-a entre colchetes.

Um array em JSON é uma coleção ordenada de valores. Ao contrário dos pares de chave-valor na raiz de um objeto JSON, os arrays JSON são ordenados, para que você possa analisá-los por valor de índice. Por exemplo, se você deseja extrair "mint" do objeto de exemplo, consulte **objectName.additional_ingredients[1]**.

Os valores de array podem ser um dos seguintes tipos: string, número, objeto, array ou booleano. O array do exemplo tem dois valores, sendo que ambos são strings (**amoras** e **menta**). No entanto, em documentos JSON mais longos e complexos, muitas vezes você verá arrays que contêm outros objetos JSON ou outros arrays JSON.

Introdução ao YAML

O que é o YAML?

YAML Ain't Markup Language (YAML)

- É uma linguagem de serialização de dados legível por humanos
 - Usado frequentemente para *arquivos de configuração*
 - Usado para descrever dados
- É projetado para ser portátil
 - Funciona bem com C, Java, Perl, Python, Ruby e outras linguagens
- Fornece recursos semelhantes ao JSON
 - Estruturas de dados complexas podem ser trocadas em um único documento.

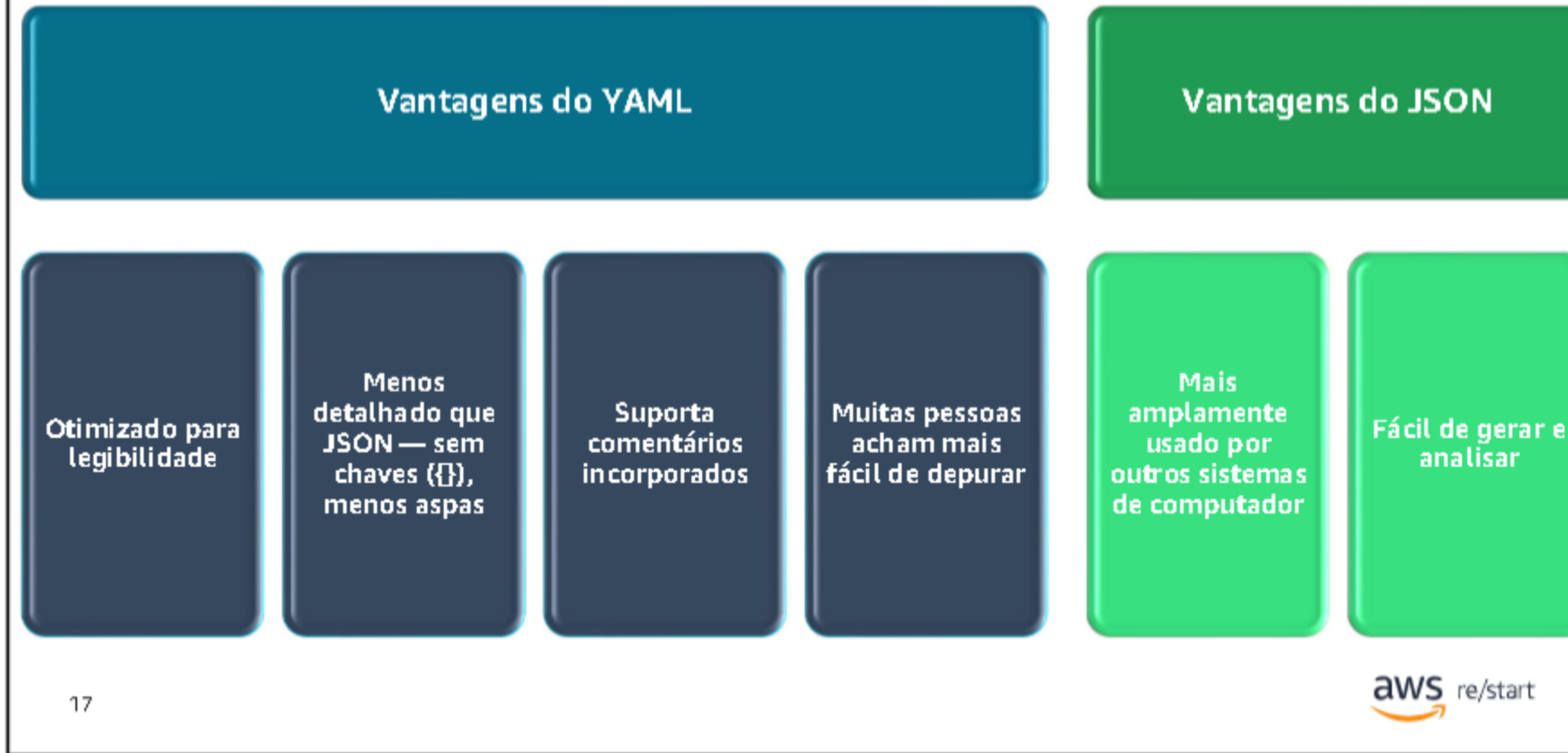
Alguns dos serviços da AWS discutidos posteriormente neste módulo, como o AWS CloudFormation, também oferecem suporte ao YAML como um formato de dados. Para usar o AWS CloudFormation, é importante entender o formato YAML. Você deve ser capaz de entender a estrutura de um documento YAML e talvez queira criar algumas seções de modelo do AWS CloudFormation no YAML. Portanto, nesta seção, você será apresentado aos fundamentos do YAML.

Eles queriam distinguir o YAML de linguagens, como Markdown e XML, que muitas vezes são usadas para marcar texto. Assim como JSON, YAML é uma sintaxe para armazenar dados. Ele é um formato baseado em texto e, portanto, é legível por humanos. Os documentos YAML são facilmente gravados. Eles

Alguns dos serviços da AWS discutidos posteriormente neste módulo, como o AWS CloudFormation, também oferecem suporte ao YAML como um formato de dados. Para usar o AWS CloudFormation, é importante entender o formato YAML. Você deve ser capaz de entender a estrutura de um documento YAML e talvez queira criar algumas seções de modelo do AWS CloudFormation no YAML. Portanto, nesta seção, você será apresentado aos fundamentos do YAML.

Eles queriam distinguir o YAML de linguagens, como Markdown e XML, que muitas vezes são usadas para marcar texto. Assim como JSON, YAML é uma sintaxe para armazenar dados. Ele é um formato baseado em texto e, portanto, é legível por humanos. Os documentos YAML são facilmente gravados. Eles armazenam pares de chave-valor, listas e arrays associativos de dados. Você pode armazenar estruturas de dados muito complexas em um único documento YAML.

YAML em comparação ao JSON



Em comparação com JSON, o YAML é otimizado para legibilidade. Ele é menos detalhado que o JSON. Os mesmos dados armazenados no JSON ocupam menos linhas para armazenar no YAML. Isso ocorre porque o YAML não usa chaves e usa menos aspas. Outra vantagem do YAML é que ele oferece suporte nativo a comentários incorporados. Por fim, muitas pessoas acreditam que é mais fácil depurar documentos YAML em comparação com JSON. Com o JSON, pode ser difícil encontrar vírgulas ou chaves ausentes ou mal posicionadas. O YAML não tem esse problema.

Apesar das muitas vantagens do YAML, o JSON oferece algumas vantagens exclusivas. Primeiro, ele é amplamente usado por sistemas de computador. A sua onipresença é uma vantagem, pois os dados armazenados no JSON podem ser

Em comparação com JSON, o YAML é otimizado para legibilidade. Ele é menos detalhado que o JSON. Os mesmos dados armazenados no JSON ocupam menos linhas para armazenar no YAML. Isso ocorre porque o YAML não usa chaves e usa menos aspas. Outra vantagem do YAML é que ele oferece suporte nativo a comentários incorporados. Por fim, muitas pessoas acreditam que é mais fácil depurar documentos YAML em comparação com JSON. Com o JSON, pode ser difícil encontrar vírgulas ou chaves ausentes ou mal posicionadas. O YAML não tem esse problema.

Apesar das muitas vantagens do YAML, o JSON oferece algumas vantagens exclusivas. Primeiro, ele é amplamente usado por sistemas de computador. A sua onipresença é uma vantagem, pois os dados armazenados no JSON podem ser usados com segurança em muitos sistemas sem precisar de transformação. Além disso, geralmente é mais fácil gerar e analisar JSON do que gerar e analisar YAML.

O AWS CloudFormation, que será discutido em detalhes posteriormente neste módulo, oferece suporte a modelos escritos no JSON ou YAML.

Noções básicas de sintaxe YAML

Alguns princípios básicos da sintaxe YAML:

- Use *recuo de espaço em branco* para indicar e criar estruturas de dados.
 - Não use guias.
 - Dois espaços é o recuo padrão.
- Adicione um *comentário* usando um sinal numérico (#) no início da linha.
- *Listas*:
 - Crie uma lista iniciando a linha com um hífen (um item da lista por linha).
 - Como alternativa, defina a lista em uma única linha, usando colchetes ([]) e vírgulas (,) entre os itens.
- Crie *matrizes associativas* usando dois pontos seguidos por um espaço.
- <key>: <value> (um item por linha)
- As *strings* normalmente não estão entre aspas.
 - No entanto, eles podem estar entre aspas duplas ou simples.

Denotando estruturas de dados

A sintaxe YAML usa recuo de espaço em branco para denotar e criar estruturas de dados. É importante saber que o YAML interpreta as guias como espaços em branco. Você deve ter cuidado para evitar o uso de tabs ao criar ou editar documentos YAML. Dois espaços em branco é o recuo padrão. No entanto, contanto que a sua abordagem de recuo seja consistente, o documento será válido.

Como adicionar comentários

Para adicionar um comentário, use um sinal numérico (#) no início da linha. Mantenha uma linha em branco antes e depois de qualquer linha comentada.

Denotando estruturas de dados

A sintaxe YAML usa recuo de espaço em branco para denotar e criar estruturas de dados. É importante saber que o YAML interpreta as guias como espaços em branco. Você deve ter cuidado para evitar o uso de tabs ao criar ou editar documentos YAML. Dois espaços em branco é o recuo padrão. No entanto, contanto que a sua abordagem de recuo seja consistente, o documento será válido.

Como adicionar comentários

Para adicionar um comentário, use um sinal numérico (#) no início da linha. Mantenha uma linha em branco antes e depois de qualquer linha comentada.

Como criar listas

Para criar uma lista, inicie a linha com um hífen e grave apenas um item de lista por linha. Como alternativa, você pode definir uma lista inteira colocando vários itens de linha em uma única linha. No entanto, para usar essa estrutura de lista, você deve colocar a lista entre colchetes e usar vírgulas entre os itens da lista.

Como criar matrizes

Para criar uma matriz associativa, use dois pontos seguidos por um espaço. Você verá um exemplo dessa sintaxe a seguir.

Definindo pares de chave-valor

Assim como JSON, os documentos YAML armazenam pares de chave-valor. Para definir um único par de chave-valor, use dois pontos entre a chave e o valor (use um item por linha).

Como criar strings

Por fim, ao contrário do JSON, as strings em YAML normalmente *não* são colocadas entre aspas. No entanto, às vezes você pode vê-las entre aspas duplas ou simples. Geralmente, as aspas são usadas para escapar caracteres que aparecem na string e que, de outra forma, confundem um sistema lendo um documento YAML.

Exemplos: JSON e YAML

JSON*

```
"WebServer": {
  "Type": "AWS::EC2::Instance",
  "Properties": {
    "ImageId": "ami-09ead922c1dad67e4",
    "InstanceType": "t2.micro",
    "KeyName": "myKey",
    "SecurityGroupIds": [
      {
        "Ref": "SecurityGroupIDs"
      }
    ],
    "SubnetId": "subnet-42b01138"
  }
}
```

*Código não real

YAML*

```
WebServer:
  Type: AWS::EC2::Instance
  Properties:
    ImageId: ami-09ead922c1dad67e4
    InstanceType: t2.micro
    KeyName: myKey
    SecurityGroupIds: !Ref SecurityGroupID
    SubnetId: subnet-42b01138
```

*Código não real

Os exemplos mostram as mesmas informações de configuração documentadas nos formatos JSON e YAML.

A versão JSON tem mais caracteres e inclui muitas aspas, chaves e vírgulas. A versão YAML tem menos caracteres e usa recuo com espaços (não tabs) para criar relacionamentos pai-filho entre partes de dados.

- O arquivo JSON usa chaves para indicar que Tipo e Propriedades são filhos do WebServer pai.

Os exemplos mostram as mesmas informações de configuração documentadas nos formatos JSON e YAML.

A versão JSON tem mais caracteres e inclui muitas aspas, chaves e vírgulas. A versão YAML tem menos caracteres e usa recuo com espaços (não tabs) para criar relacionamentos pai-filho entre partes de dados.

- O arquivo JSON usa chaves para indicar que Tipo e Propriedades são filhos do WebServer pai.
- O arquivo YAML usa recuo de espaço em branco para indicar que Tipo e Propriedades são filhos do WebServer pai.

Observe a diferença em como os pares de chave-valor são documentados. No YAML, a chave e o valor *não* estão entre aspas. Os finais das linhas não têm vírgulas, e as chaves e os valores não precisam estar entre aspas.

Agora você tem um entendimento geral da sintaxe do JSON e do YAML. Esse entendimento deve ajudá-lo enquanto você começa a explorar os recursos do AWS CloudFormation neste módulo.

Principais conclusões



© 2020, Amazon Web Services, Inc. ou suas afiliadas. Todos os direitos reservados.

20

- JSON
 - Sintaxe para armazenar e transportar dados.
 - Formato baseado em texto, portanto, é legível por humanos.
 - Os documentos são escritos facilmente.
 - Armazena pares de valores-chave e matrizes de dados.
- YAML
 - Sintaxe para armazenar dados.
 - Formato baseado em texto, portanto, é legível por humanos.
 - Os documentos são escritos facilmente.
 - Armazena pares de valores-chave, listas e matrizes associativas de dados.
 - Armazena estruturas de dados complexas em um único documento YAML.

aws re/start

- JSON
 - Sintaxe para armazenar e transportar dados.
 - Formato baseado em texto, portanto, é legível por humanos.
 - Os documentos são escritos facilmente.
 - Armazena pares de valores-chave e matrizes de dados.
- YAML
 - Sintaxe para armazenar dados.
 - Formato baseado em texto, portanto, é legível por humanos.